FORMATO DE DESARROLLO E INTEGRACIONES

API INSPEKTOR

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, martes, 15 de septiembre de 2026

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios.....**¡Error! Marcador no definido.**
2. CASOS DE PRUEBAS UNITARIAS 12

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

En el presente documento se describe técnicamente como fue desarrollada la API que interviene en el proceso de originación de cupo para Credi 10. Los pasos claves de este requerimiento incluye:

a) Consumo del servicio web Inspektor: La API se conecta de manera segura al servicio proporcionado por INSPEKTOR donde se obtendrá información de los clientes en listas que ellos administran.
b) Procesamiento de datos: Una vez se recibe los datos del servicio web se lleva a cabo una normalización de la información recibida y el procesamiento de esta para otorgar una decisión de negación del cupo o de continuar con el estudio.
c) Retorno de información por listas restrictivas: La API responde a las solicitudes de los usuarios proporcionando la información de si el cliente se encuentra o no en listas restrictivas.

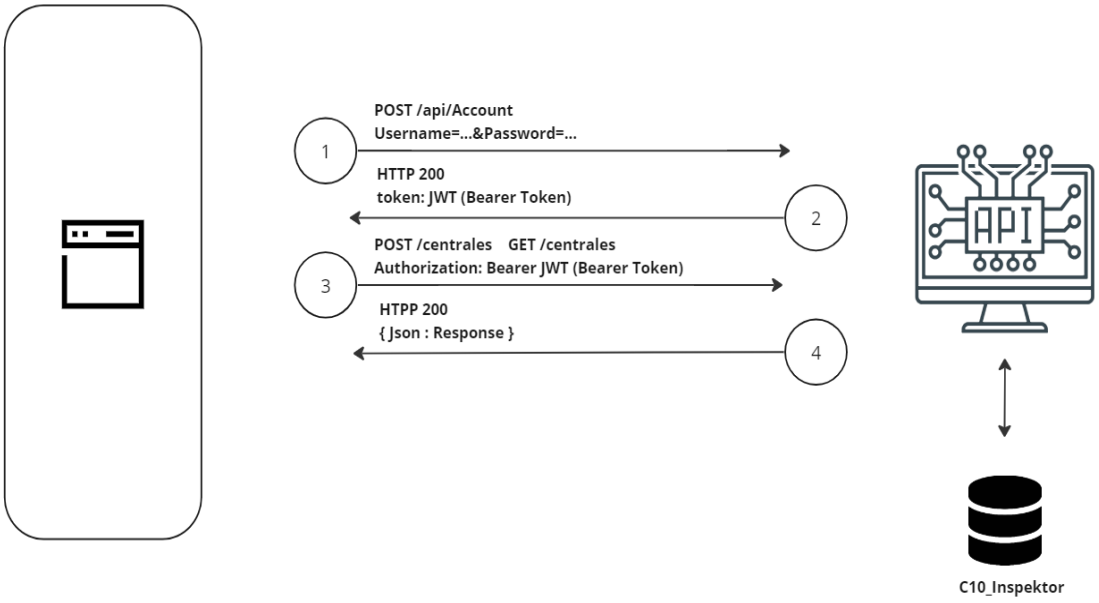
1.2 Detallar componentes existentes

SOLUCION	
Inspektor.Integrator.Api	Proyecto que proporciona un conjunto de servicios a otras aplicaciones a través de una interfaz de programación de aplicaciones (API).
Inspektor.Integrator.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de consumir el servicio web, procesarlo y entregar la información para su posterior envió al usuario final. Este proyecto se integra con el proyecto API y el proyecto DAL.
Inspektor.Integrator.DAL	Proyecto encargado de tener el acceso y la interacción a la base de datos C10_Inspektor de la solución
Web Service Inspektor	URL CONSUMO: https://inspektor.datalaft.com:2055/WSInspektor.asmx URL WSDL: https://inspektor.datalaft.com:2055/WSInspektor.asmx?WSDL

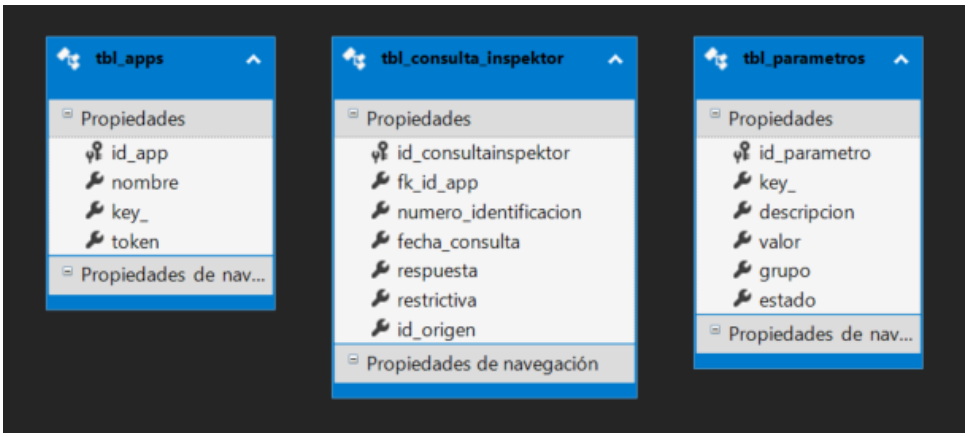
BASE DE DATOS	
C10_Inspektor.dbo.tbl_apps	Tablas
C10_Inspektor.dbo.tbl_consulta_inspektor	Tablas
C10_Inspektor.dbo.tbl_parametros	Tablas

1.3 Detallar funcionalidades

1.3.1 Diagrama de Arquitectura



1.3.2 MER



1.3.3 Parametrizaciones

La aplicación cuenta con 6 parámetros, estos no tienen una distinción en el ambiente de pruebas y producción en lo que corresponde el consumo al web service de Inspektor estos no dieron un ambiente de prueba para consumir.

Llave parametros	**Descripcion**	**Valor ambiente productivo**	**Valor ambiente pruebas**
InspektorURL	URL servicio Inspektor	https://inspektor.datalaft.com:2055/WSInspektor.asmx	https://inspektor.datalaft.com:2055/WSInspektor.asmx
ServicePassword	Contraseña del cliente Inspektor	xxx	xxx
JWT_SECRET_KEY	Llave secreta para el JWT	xxxxxxx	xxxxx
JWT_AUDIENCE_TOKEN	Cliente con autorizacion para consumir la API	http://192.168.0.xxx/	http://192.168.0.xxx/
JWT_ISSUER_TOKEN	Cliente que podra realizar peticiones a la API	http://192.168.0.xxx/	http://192.168.0.xxx/
JWT_EXPIRE_MINUTES	Tiempo de expiracion para el consumo de la API	5	5

1.3.4 Proceso

El primer paso para poder consumir la API consiste en la autenticación mediante una solicitud POST. Este proceso implica recibir un par de credenciales de usuario, compuesto por el nombre de usuario (username) y la contraseña (password). Si las credenciales proporcionadas son válidas, la API genera y devuelve un token de acceso JSON Web Token (JWT). Este token se utiliza posteriormente para autorizar y autenticar las solicitudes subsiguientes del usuario. En caso de que las credenciales recibidas en la solicitud sean inválidas, la API responde con un código de estado HTTP 401 (No Autorizado), indicando que la autenticación ha fallado. Este mecanismo de autenticación basado en JWT contribuye a la seguridad de la API al proporcionar un método eficaz para la gestión de sesiones y la protección de recursos mediante la verificación de la identidad del usuario.

AccountController.cs

Inspektor.Integrator.API

```
using System.Web.Http;
using Inspektor.Integrator.BLL.DTO;
using Inspektor.Integrator.BLL.Logic;

namespace Inspektor.Integrator.API.Controllers
{
    [AllowAnonymous]
    0 referencias
    public class AccountController : ApiController
    {
        /// <summary>
        /// Método encargado de realizar la autenticación
        /// </summary>
        /// <param name="loginDTO"></param>
        /// <returns></returns>
        [HttpPost]
        0 referencias
        public IHttpActionResult Login(AccountDTO loginDTO)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            bool isCredentialValid = Jwt.VerifyCredentials(loginDTO.Username, loginDTO.Password);

            if (isCredentialValid)
            {
                var response = new SecurityResponse
                {
                    status = true,
                    token = TokenGenerator.GenerateTokenJwt(loginDTO.Username)
                };
                return Ok(response);
            }
            else
            {
                return Unauthorized(); //Status code 401
            }
        }
    }
}
```

En la verificación de las credenciales recibidas la aplicación consulta si el usuario existe en la base de datos de la aplicación y si la contraseña recibida pertenece a dicho usuario. Si las credenciales son válidas se inicia el proceso de generar el JWT para su posterior envió como respuesta a la solicitud.

```
internal static class TokenGenerator
{
    /// <summary>
    /// Metodo para generar el token JWT que dara permiso para consumir las APIs con este requisito
    /// </summary>
    /// <param name="username"></param> nombre de usuario para tener la identidad del usuario
    /// <returns></returns>
    1 referencia
    public static string GenerateTokenJwt(string username)
    {
        // Configuración de la aplicación para el Token JWT
        var secretKey = ParameterManager.GetParam(Constants.ParametrosJWT.SecretKey);
        var audienceToken = ParameterManager.GetParam(Constants.ParametrosJWT.AudienceToken);
        var issuerToken = ParameterManager.GetParam(Constants.ParametrosJWT.IssuerToken);
        var expireTime = ParameterManager.GetParam(Constants.ParametrosJWT.ExpireTime);

        var securityKey = new SymmetricSecurityKey(System.Text.Encoding.Default.GetBytes(secretKey));
        var signingCredentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256Signature);

        // Crear un claimsIdentity
        ClaimsIdentity claimsIdentity = new ClaimsIdentity(new[] { new Claim(ClaimTypes.Name, username) });

        // Crear token al usuario
        var tokenHandler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
        var jwtSecurityToken = tokenHandler.CreateJwtSecurityToken(
            audience: audienceToken,
            issuer: issuerToken,
            subject: claimsIdentity,
            notBefore: DateTime.UtcNow,
            expires: DateTime.UtcNow.AddMinutes(Convert.ToInt32(expireTime)),
            signingCredentials: signingCredentials);

        var jwtTokenString = tokenHandler.WriteToken(jwtSecurityToken);
        return jwtTokenString;
    }
}
```

El método *GenerateTokenJwt* funciona de la siguiente manera:

- Obtención de Parámetros: Se obtienen los parámetros necesarios para la creación del JWT desde la configuración de la aplicación. Estos parámetros incluyen la clave secreta (secretKey), la audiencia (audienceToken), el emisor (issuerToken) y el tiempo de expiración (expireTime), más adelante se detallará cada uno de estos parámetros.

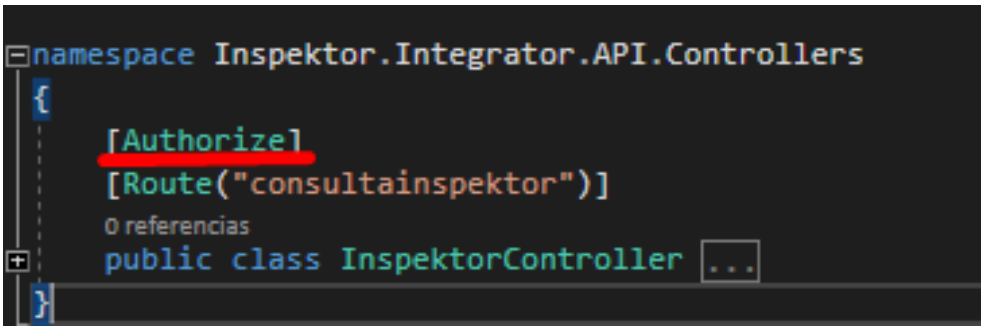
- Creación de la Clave de Seguridad: Se crea una clave de seguridad (securityKey) utilizando la clave secreta que se encuentra parametrizada en base de datos. Esta clave se utiliza para firmar el token y verificar su autenticidad posteriormente.

- Creación de Credenciales de Firma: Se crean las credenciales de firma (signingCredentials) utilizando la clave de seguridad y el algoritmo de firma HmacSha256. Estas credenciales se utilizan al generar la firma del token.

- Creación de ClaimsIdentity: Se crea un ClaimsIdentity que representa la identidad del usuario asociado al token. En este caso, se incluye un claim que contiene el nombre de usuario. **Nota:** *Un claim es una afirmación sobre el usuario que se agrega al token.*

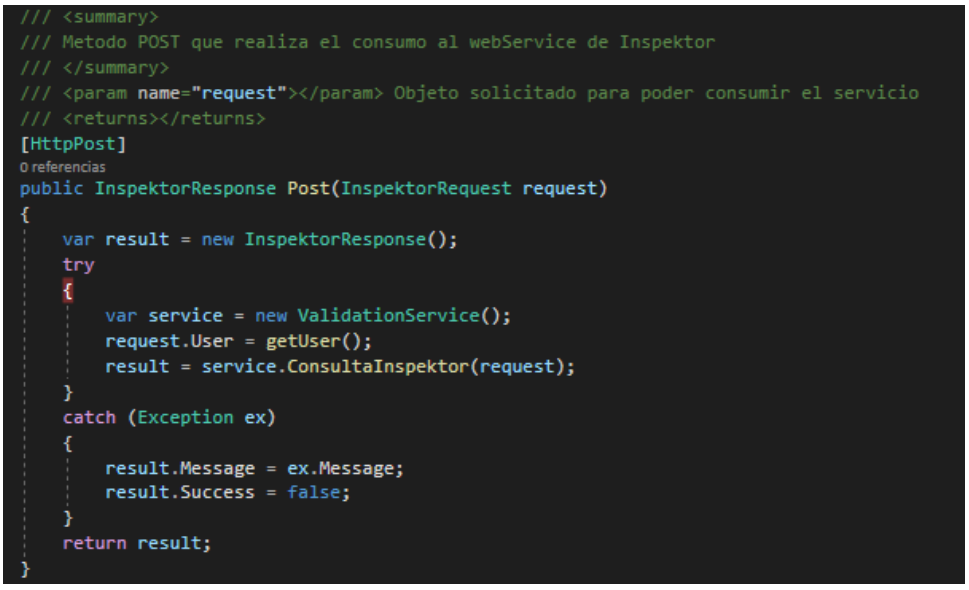
Estas líneas de código son esenciales para la creación segura de un JWT, ya que establecen los parámetros clave, la clave de seguridad para la firma, las credenciales de firma y la identidad del usuario asociada al token. Esta funcionalidad es crucial para garantizar la autenticidad y la integridad del JWT generado.

Si el JWT token fue creado correctamente este se envía al usuario como respuesta para el posterior uso de este. La API fue desarrollada para que todas las solicitudes que tuvieran la ruta */ consultainspektor* se deba recibir el token de autorización en la solicitud.



```
namespace Inspektor.Integrator.API.Controllers
{
    [Authorize]
    [Route("consultainspektor")]
    0 referencias
    public class InspektorController ...
}
```

De esta manera cada vez que se recibe una solicitud, GET o POST, este pasará por el filtro de Authorize que será el encargado de hacer ingeniería inversa y validar que el JWT recibido este creado por la misma aplicación y el tiempo de expiración no allá excedido su límite.



```
/// <summary>
/// Metodo POST que realiza el consumo al webService de Inspektor
/// </summary>
/// <param name="request"></param> Objeto solicitado para poder consumir el servicio
/// <returns></returns>
[HttpPost]
0 referencias
public InspektorResponse Post(InspektorRequest request)
{
    var result = new InspektorResponse();
    try
    {
        var service = new ValidationService();
        request.User = getUser();
        result = service.ConsultaInspektor(request);
    }
    catch (Exception ex)
    {
        result.Message = ex.Message;
        result.Success = false;
    }
    return result;
}
```

El método POST de la aplicación es la encargada de consumir el servicio web de Inspektor, la clase de *service.ConsultaInspektor s*erá la encargada de configurar la conexión con el servicio web, hacer el respectivo consumo para obtener las listas restrictivas y procesar la información obtenida para enviarla como respuesta del API. Esta tambien se guardará en la tabla llamada [*tbl_consulta_inspektor]* con la respuesta encriptada.

Script para ver las consultas guardadas hechas por el API.

```
SELECT TOP (5) [id_consultainspektor]
    ,[fk_id_app]
```

```
    ,[numero_identificacion]
    ,[fecha_consulta]
    ,[respuesta]
    ,[restrictiva]
    ,[id_origen]
FROM [C10_Inspektor].[dbo].[tbl_consulta_inspektor]
```

	id_consultainspektor	fk_id_app	numero_identificacion	fecha_consulta	respuesta	restrictiva	id_origen
1	577912	1	1071549774	2023-12-14 21:54:43.757	2+0VIW1Ksx4d2Rjz2FCkLRmGrziFPXQVkFEgtO/rHSnx0a6Ff...	0	NULL
2	577911	1	1040751560	2023-12-14 21:51:52.747	2+0VIW1Ksx4d2Rjz2FCkLRmGrziFPXQVkFEgtO/rHSnx0a6Ff...	0	NULL
3	577910	1	1140848280	2023-12-14 21:43:48.723	2+0VIW1Ksx4d2Rjz2FCkLRmGrziFPXQVkFEgtO/rHSnx0a6Ff...	0	NULL
4	577909	1	86065575	2023-12-14 21:43:34.743	2+0VIW1Ksx4d2Rjz2FCkLRp7+hosnoE8kWSgiEWqwjmfa57z...	1	NULL
5	577908	1	52125922	2023-12-14 21:43:18.797	2+0VIW1Ksx4d2Rjz2FCkLRmGrziFPXQVkFEgtO/rHSnx0a6Ff...	0	NULL

Al saber que la respuesta obtenida son datos sensibles de los prospectos de Credi10, se decidió cifrar la respuesta mediante la encriptación AES 256 que es la encargada de proteger la información utilizando una clave de 256 bits para cifrar y descifrar la información.

```
/// <summary>
/// Metodo para obtener las diferentes consultas realizadas a un numero de identificacion
/// </summary>
/// <param name="id"></param> Numero de identificacion del cliente
/// <returns></returns>
[HttpGet]
[Route("consultainspektor/{id}")]
0 referencias
public List<ConsultaResponse> Get(string id)
{
    var bo = new BussinesLog();
    return bo.List(id);
}
```

El método GET de la aplicación recibe en su solicitud un id que será igual a el número de identificación de la persona a la que quiere consultar, este método devolverá todas las solicitudes realizadas y guardadas en la base de datos de la persona con información de las listas restrictivas que tuvo en cada consulta.

1.4 Lista de Programas, Archivos, Paquetes, Elementos BD

[C10_Inspektor].[tbl_apps]

Tabla dentro del esquema C10_Inspektor, el cual permite almacenar los usuarios que serán autorizados para poder consumir los métodos del API.

[C10_Inspektor].[tbl_consulta_inspektor]

Tabla dentro del esquema C10_ Inspektor, el cual permite almacenar todas las solicitudes realizadas a la API y la respuesta que obtuvo cada solicitud realizada.

[C10_Inspektor].[tbl_parametros]

Tabla dentro del esquema C10_ Inspektor, el cual permite almacenar y obtener en la aplicación los diferentes parámetros que permiten el correcto consumo al servicio web de Inspektor.

1.5 Lista de Script's y la descripción de lo que contiene cada uno

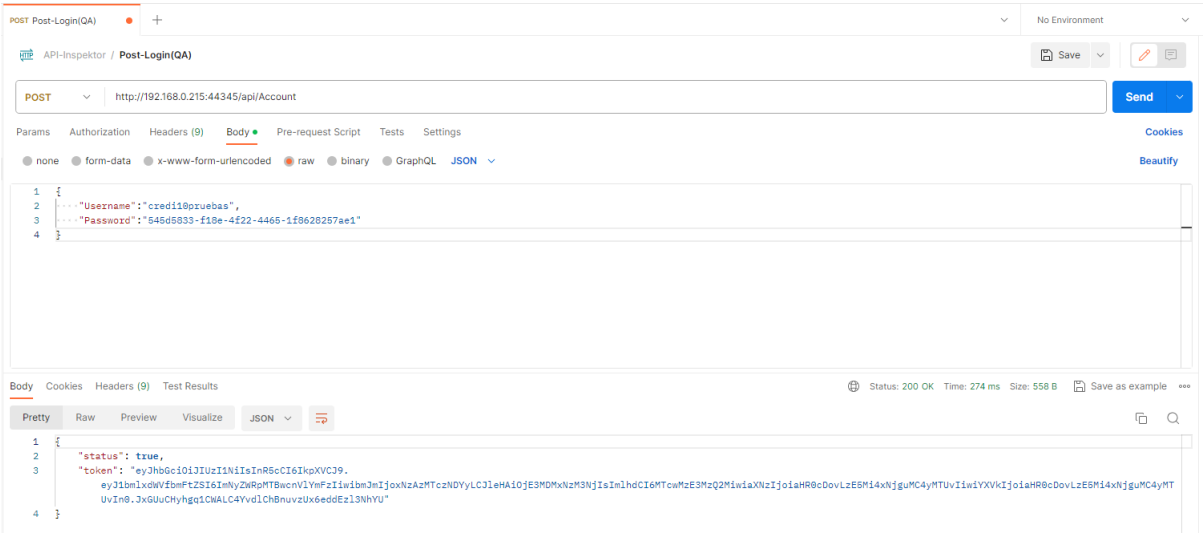
Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Originacion_API_Inspektor_Creacion_Tablas.sql

2. CASOS DE PRUEBAS UNITARIAS

A continuación, se demuestra las pruebas funcionales hechas desde POSTMAN.

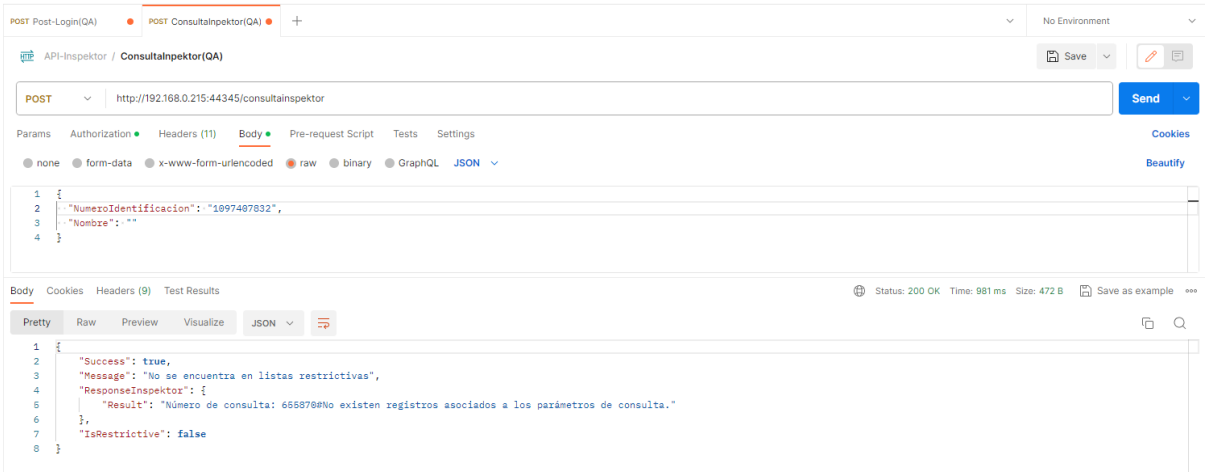
1. Primero se debe realizar el consumo para autenticar el usuario con el API. La solicitud debe ser hecha mediante POST, y en el cuerpo de la solicitud se debe enviar un username y password que serán dados a los usuarios (diferentes aplicaciones) para autorizar su consumo.



El curl de la solicitud es el siguiente:

```
curl --location 'http://192.168.0.xxx:xxxxx/api/Account' \
--header 'Content-Type: application/json' \
--data '{
    "Username":"xxxxx",
    "Password":"xxxxx"
}'
```

2. Si en la primera solicitud nos da un status 200, nos devolverá un token que será el que deberá utilizar para las siguientes solicitudes que se quieran realizar. En este caso se hará el consumo del POST que es el que consumirá el servicio web de inspektor.



En el cuerpo de la solicitud debe ir la información respectiva del cliente a quien se le quiere hacer la consulta. En la respuesta se nos detallara si se encuentra o no en alguna lista restrictiva.

El curl de la solicitud es el siguiente:

curl --location 'http://192.168.0.xxx:xxxxx/consultainspektor' \
--header 'app-user: xxx \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImNyZWRpMTBwcnVlYmFzIiwibmJmIjoxNzAzMTczNDYyLCJleHAiOjE3MDMxNzM3NjIsImlhdCI6MTcwMzE3MzQ2MiwiaXNzIjoiaHR0cDovLzE5Mi4xNjguMC4yMTUvIiwiYXVkIjoiaHR0cDovLzE5Mi4xNjguMC4yMTUvIn0.JxGUuCHyhgq1CWALC4YvdlChBnuvzUx6eddEzl3NhYU' \
--data '{
"NumeroIdentificacion": "1097407832",
"Nombre": ""
}'

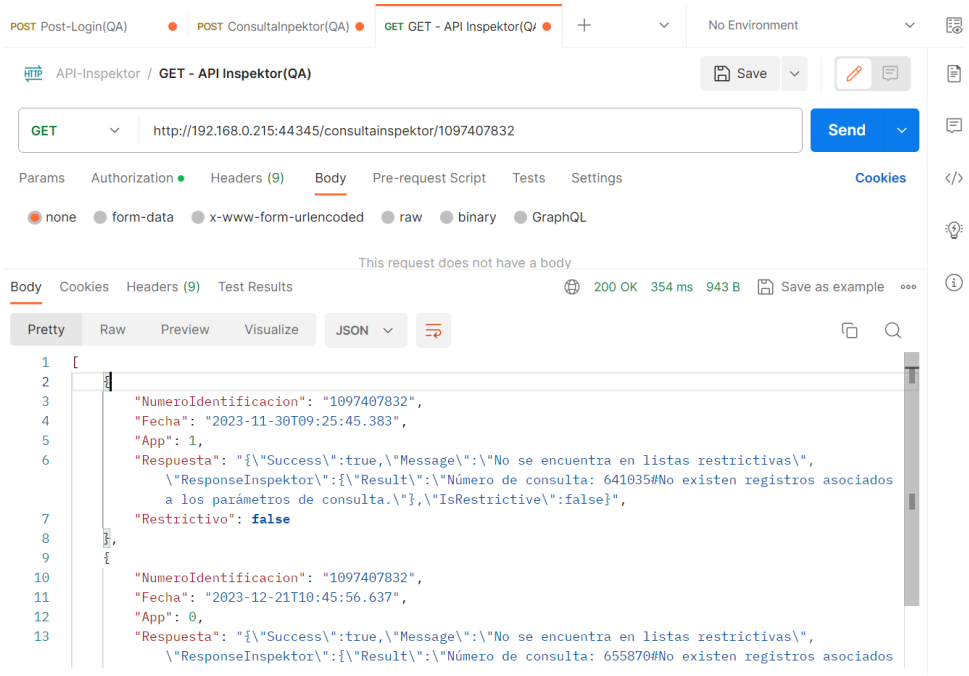
Interpretación respuesta

A continuación, se describen los campos o columnas presentadas en el json de respuesta

- Success: Boleano que identifica si el proceso de consulta finalizo correctamente.
- Message: Devuelve mensaje informativo de la respuesta.
- IsRestrictive: Booleano que identifica si la cedula evaluada cumple la condición de si es restrictivo o no para continuar el estudio crediticio para el cupo de credi10.

- ResponseInspektor: Objeto que devuelve inspektor con el resultado de la consulta.

3. La otra solicitud disponible es un GET el cual consulta todas las solicitudes hechas mediante el API a una cedula determinada.



El curl de la solicitud es el siguiente:

curl --location 'http://192.168.0.xxx:xxxxx/consultainspektor/1097407832' \
--header 'app-user: credi10web' \
--header 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1bmlxdWVfbmFtZSI6ImNyZWRpMTBwcnVlYmFzIiwibmJmIjoxNzAzMTczODUwLCJleHAiOjE3MDMxNzQxNTAsImlhdCI6MTcwMzE3Mzg1MCwiaXNzIjoiaHR0cDovLzE5Mi4xNjguMC4yMTUvIiwiYXVkIjoiaHR0cDovLzE5Mi4xNjguMC4yMTUvIn0.S3nSV62uzlR8kTRObYYXY0cLyXnzQjKFQT_GYk7sjLA'

ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Autenticacion al API	Devolución de un JWT	SI
2	Consultar listas restrictivas con una cedula	Devolucion lista	SI
3	Consulta de solicitudes a una cedula	Devolucion lista de solicitudes realizadas	SI

Recurso para realizar pruebas por Postman

- Originacion_API_Inspektor.postman_collection